Document Number: P1423R3
Date: 2019-07-18
Audience: Library Evolution Working Group
Library Working Group
Reply-to: Tom Honermann <tom@honermann.net>

char8_t backward compatibility remediation

Changes since P1423R2

- Added links to a github repository that implements several of the remediation approaches discussed in this paper.
- Updated the "Emulate C++17 `u8` literals" remediation approach with corrections regarding P0388R2 in C++20 (it has not yet been adopted for C++20), and current limitations using the example code with gcc 9.1.0.
- Corrected several link titles for references to P0388R2.
- Updated the "Add Overloads" remediation approach to discuss ambiguous overload resolution issues based on feedback provided by Marc Mutz. Thanks, Marc!
- Rebased the proposed wording on N4820.

Introduction

The support for `char8_t` as adopted for C++20 via P0482R6 [P0482R6] affects backward compatibility for existing C++17 programs in at least the following ways:

1. Introduction of a new `char8_t` keyword, new `std::u8string`, `std::u8string_view`, `std::u8streampos` type aliases and `std::mbrtoc8` and `std::c8rtomb` functions; these names may conflict with existing uses of these names.
2. Change of return type for `std::filesystem::path` member functions `u8string` and `generic_u8string`.
3. Change of type for `u8` character and string literals and `u8R` raw string literals.

This paper does *not* further discuss case 1 above. Adding new keywords and new members to the `std` namespace is business as usual; see SD-8 [SD-8]. It is acknowledged that these additions will affect some code bases. Code surveys have found that these names have generally been used to emulate the set of features introduced with the adoption of P0482R6 [P0482R6]. In some cases, existing code has already been updated to adapt to the new standard features. For example, EASTL will now use the the standard provided `char8_t` type when available instead of the type alias previously used. The pull request for this change can be found at https://github.com/electronicarts/EASTL/pull/239.

Case 2 above is a change that does *not* fit into the set of standard library rights reserved in SD-8 [SD-8]. This is a cause for concern, but is somewhat mitigated by the fact that `std::filesystem` is new with C++17 and therefore does not have a long history of use. Some options for dealing with this change are discussed later in this paper.

Case 3 above is the change responsible for most of the backward compatibility impact.

This paper is motivated by three goals:

- To document a set of options available to programmers to facilitate migration of existing code to C++20. Where possible, options are presented for writing code that is compatible with both C++17 and C++20.
- To ensure that WG21 members are aware of the backward compatibility issues and anticipated impact, and find the set of options available to mitigate the impact acceptable.
- To consider options available to reduce backward compatibility impact. This paper documents a number of such options, but only proposes two small standard library changes intended to remove backward compatibility impact that was not intended by the adoption of P0482R6.

Examples

The following table presents examples of well-formed C++17 code that is either ill-formed or behaves differently in C++20. The table also reflects the intended changes proposed in this paper. Note that most of these examples remain ill-formed with this proposal. This is intentional as the examples reflect problematic code that leads to mojibake in C++17 code due to use of the same type (`char`) for multiple encodings (execution encoding and UTF-8).

Code	C++17	C++20 with P0482R6	C++20 with this proposal
`const char *p = u8"text";`	Initializes `p` with the address of the UTF-8 encoded string.	Ill-formed.	Ill-formed.
`char a[] = u8"text";`	Initializes `a` with the UTF-8 encoded string.	Ill-formed.	Ill-formed.
`int operator ""_udl(const char*, unsigned long);` `int v = u8"text"_udl;`	Initializes `v` with the result of calling `operator ""_udl` with the UTF-8 encoded string literal.	Ill-formed.	Ill-formed.
`std::string s(u8"text");`	Initializes `s` with the UTF-8 encoded string.	Ill-formed.	Ill-formed.
`std::filesystem::path p = ...;` `std::string s = p.u8string();`	Initializes `s` with the UTF-8 encoded representation of the file path stored in `p`.	Ill-formed.	Ill-formed.
`std::cout << u8'x';` `std::cout << u8"text";`	Writes a sequence of UTF-8 code units as characters to stdout. (mojibake if the execution character encoding is not UTF-8)	Writes an integer or pointer value to stdout. (consistent with handling of char16_t and char32_t)	Ill-formed. (for all of char8_t, char16_t, and char32_t)
`std::filesystem::u8path(u8"filename");`	Constructs a `std::filesystem::path` object from the UTF-8 encoded string.	Ill-formed.	Constructs a `std::filesystem::path` object from the UTF-8 encoded string.

Anticipated impact

`u8` string literals were added in C++11, but support for `u8` character literals was only added in C++17.

Code surveys have so far revealed little use of `u8` literals. Google and Facebook have both reported less than 1000 occurrences in their code bases, approximately half of which occur in test code. Representatives of both organizations have stated that, given the actual size of their code base, this is an insignificant number of occurrences. Likewise, Firefox contains around 100 occurrences and Mozilla engineers have reviewed this paper and have no concerns regarding addressing the existing uses using the remediation approaches discussed here.

Code surveys have been attempted on github, but github search doesn't facilitate distinguishing uses of `u8` as identifiers (which is quite common) vs use as a UTF-8 literal. Further, github doesn't provide a search that filters out duplicate hits for the same source code in different repositories. As a result, finding instances of `u8` literals is challenging. Most cases that were identified were in tests included in clones of Clang and gcc.

Searches on Debian code search found uses in only a few packages and, with one exception discussed below, only a small number of uses (mostly single digit counts), most of which occurred in tests.

Survey results for Debian code search (https://codesearch.debian.net) follow. These exclude hits to gcc, clang, packages like fmt and eastl that have already been conditionalized for `char8_t` in C++20, and C code. Additionally, packages bundled with other packages are omitted to avoid double counting. Search hits reflect number of lines matching the search, not number of occurrences. Search results are categorized by hits in program source files vs hits in test source files.

There is one clear outlier in these search results. Chromium has two orders of magnitude more uses of `u8` literals than any other package. The Chromium team was contacted to discuss this. Almost all of the existing uses appear in bulk data source files that the Chromium team has determined can be mechanically addressed.

Searched for	Debian packages (out of ~18000)
`char8_t`	spring (emulates its own `char8_t` support)
`u8string`	libopenmpt (defines a `mpt::u8string` typedef of `std::basic_string` with a custom `char_traits`) spring (defines a `std::u8string` class that derives from `std::string`)
`u8string_view`	<none>
`u8streampos`	<none>
`mbrtoc8`	<none>
`c8rtomb`	<none>
`u8path(u8"text")`	<none>
`u8"text"`	chromium (~10310, ~102 files) firefox (97 hits, 1 files, 3 test files) icu (83 hits, 1 files, 5 test files) qbs (56 hits, 1 file, 1 test file) mongodb (30 hits, 2 test files) aseba (28 hits, 1 file) monero (26 hits, 1 file, 1 test file) nlohmann-json (21 hits, 1 file, 3 test files)

	bambootracker (20 hits, 5 files) capnproto (18 hits, 1 file, 1 test file) lgogdownloader (11 hits, 1 file) libosmium (10 hits, 3 test files) cbmc (8 hits, 3 test files) maim (8 hits, 1 file) octave-ltfat (8 hits, 1 file) praat (8 hits, 2 files) slop (8 hits, 1 file) mame (7 hits, 3 files) nlohmann-json3 (7 hits, 3 test files) sdcc (7 hits, 1 test file) antlr4 (3 hits, 1 file) keyman-keyboardprocessor (3 hits, 2 test files) scram (3 hits, 2 test files) tesseract (3 hits, 1 test file) boost1.67 (2 hits, 1 test file) freeorion (2 hits, 1 file) supertux (2 hits, 1 file) cjs (1 hit, 1 test file) cpp-hocon (1 hit, 1 test file) efl (1 hit, 1 file) gjs (1 hit, 1 test file) kate4 (1 hit, 1 example file) kodi (1 hit, 1 test file) libtcod (1 hit, 1 test file) retroarch (1 hit, 1 file) rtags (1 hit, 1 test file)
`u8R"(text)"`	chromium (940, 2 files, 2 test files) kate4 (2 hits, 1 example file, 1 test file) nlohmann-json (2 hits, 1 test file) nlohmann-json3 (2 hits, 1 test file) ksyntax-highlighting (1 hit, 1 test file) ktexteditor (1 hit, 1 test file)

Remediation approaches

A single approach to addressing backward compatibility impact is unlikely to be the best approach for all projects. This section presents a number of options to address various types of backward compatibility impact. In some cases, the best solution may involve a mix of these options.

Each of these approaches assumes a requirement for continued use of UTF-8 encoded literals with `char` based types. For most projects, such a requirement is expected to be temporary while the project is fully migrated to C++20. However, some projects may retain a sustained need for such literals. For those projects, the Emulate C++17 `u8` literals approach is able to address most cases of backward compatibility impact.

Several of these approaches have been implemented in a header only library available at https://github.com/tahonermann/char8_t-remediation. The header provides compiler feature testing, types, and macro definitions that enable code to be written with `char` typed UTF-8 literals compatibility for C++17 and C++20.

Disable `char8_t` support

The simplest possible solution in the short term is to simply disable the new features completely. Clang and gcc will allow disabling `char8_t` features in both the language and standard library, via a `-fno-char8_t` option. It is expected that Microsoft and EDG based compilers will offer a similar option.

This option should be considered a short-term solution to enable testing existing C++17 code compiled as C++20 with minimal effort. This isn't a viable long-term option as continued use would potentially complicate composition with code that depends on the new features.

Add overloads

Adding function overloads that accept `char8_t` based types is an effective step towards full migration to C++20. Ideally, older `char` based functions would eventually be removed.

If the overlaods do not need to accept null values (`nullptr`, `0`, `NULL`), then adding a regular overload suffices as in the following example. However, this approach will result in overload resolution failure if callers are permitted to pass null values.

Before	After

`int ft(const char*);`
 	
`ft(u8"text");`
	`int ft(const char*);`
	`#if defined(__cpp_char8_t)`
	`int ft(const char8_t*);`
	`#endif`
	
	`ft(u8"text"); // C++17 or C++20`

`int operator "" _udl(const char*, unsigned long);`
 	
`int v = u8"text"_udl;`
	`int operator "" _udl(const char*, unsigned long);`
	`#if defined(__cpp_char8_t)`
	`int operator "" _udl(const char8_t*, unsigned long);`
	`#endif`
	
	`int v = u8"text"_udl; // C++17 or C++20`

If handling of null values is required, then some additional finessing is required. There are three approaches:

- Have callers cast null values to the type needed for the desired overload.
  This approach requires updates to existing callers.
- Add additional overloads for `nullptr_t` and `int`.
  This doesn't scale if there are multiple parameters.
- Define the `char8_t` overload as a template.

The third option relies on overload resolution preferring a non-template function over a function template to avoid ambiguous overload resolution as shown in the following example.

```
int ft(const char*, const char*);       // #1
#if defined(__cpp_char8_t)
template<typename = void>
int ft(const char8_t*, const char8_t*); // #2
#endif

ft(nullptr,  nullptr);  // Calls #1
ft("text",   nullptr);  // Calls #1
ft(u8"text", nullptr);  // Calls #2
ft(u8"text", "text");   // ill-formed
```

Change `u8` literals to ordinary literals with escape sequences

This approach may be a reasonable option when the execution encoding is ASCII based (but not UTF-8; otherwise just use ordinary literals) and characters outside the basic source character set are infrequently used in existing `u8` literals. This approach matches how code using UTF-8 had to be written prior to C++11.

Before	After
`u8"\u00E1"`	`"\xC3\xA1" // U+00E1`
`u8"á"` `(assuming source encoding is UTF-8)`	`"\xC3\xA1" // U+00E1` `(works with any source encoding)`

reinterpret_cast `u8` literals to `char`

Common uses of `u8` literals can be handled in a backward compatible manner through use of `reinterpret_cast`. Note that use of `reinterpret_cast` is well-formed in these situations since lvalues of type `char` may be used to access values of other types. Such code is valid in both C++17 and C++20.

This approach may suffice when there are just a few uses of UTF-8 literals that need to be addressed and the uses do not appear in `constexpr` context. In general, sprinkling `reinterpret_cast` all over a code base is not desirable.

Before	After
`const char &r = u8'x';`	`const char &r = reinterpret_cast<const char &>(u8'x');  // C++17 or C++20`
`const char *p = u8"text";`	`const char *p = reinterpret_cast<const char *>(u8"text");  // C++17 or C++20`

Emulate C++17 `u8` literals

The techniques applied here are also applicable to the examples illustrated in the prior section regarding use of `reinterpret_cast`. This approach makes use of P0732R2 [P0732R2] to enable constexpr UTF-8 encoded `char` based literals using a user defined literal. The example code below defines overloaded character and string UDL operators named `_as_char`. These UDLs can then be used in place of existing UTF-8 character and string literals.

As of gcc 9.1.0, the following code is not accepted. There are two issues:

- gcc does not yet implement support for `operator <=>`.
- gcc rejects the use of class template argument deduction (CTAD) for the `char8_t_string_literal` template parameter for the literal operator template. This issue is tracked by https://gcc.gnu.org/bugzilla/show_bug.cgi?id=88095 and a patch to gcc is available.

Applying the patch and commenting out the declaration of `operator <=>` suffices for gcc 9.1.0 to accept the code.

```
#include <utility>

template<std::size_t N>
struct char8_t_string_literal {
  static constexpr inline std::size_t size = N;
  template<std::size_t... I>
  constexpr char8_t_string_literal(
    const char8_t (&r)[N],
    std::index_sequence<I...>)
  :
    s{r[I]...}
  {}
  constexpr char8_t_string_literal(
```

```
    const char8_t (&r)[N])
  :
    char8_t_string_literal(r, std::make_index_sequence<N>())
  {}
  auto operator <=>(const char8_t_string_literal&) = default;
  char8_t s[N];
};

template<char8_t_string_literal L, std::size_t... I>
constexpr inline const char as_char_buffer[sizeof...(I)] =
  { static_cast<char>(L.s[I])... };

template<char8_t_string_literal L, std::size_t... I>
constexpr auto& make_as_char_buffer(std::index_sequence<I...>) {
  return as_char_buffer<L, I...>;
}

constexpr char operator ""_as_char(char8_t c) {
  return c;
}

template<char8_t_string_literal L>
constexpr auto& operator""_as_char() {
  return make_as_char_buffer<L>(std::make_index_sequence<decltype(L)::size>());
}
```

Before	After
`constexpr const char &r = u8'x';`	`constexpr const char &r = u8'x'_as_char;  // C++20 only`
`constexpr const char *p = u8"text";`	`constexpr const char *p = u8"text"_as_char;  // C++20 only`

`// Standard C++ doesn't permit conversion to arrays of unknown,`
`// bound, but versions of gcc prior to 9.0.1 did.`
`constexpr const char (&r)[] = u8"text";`
	`// If P0388R2 [P0388R2] is adopted, then this will be ok.`
	
	`constexpr const char (&r)[] = u8"text"_as_char; // C++20 with P0388R2`

When wrapped in macros, the above UDL can be used to retain source compatibility across C++17 and C++20 for all known scenarios except for array initialization.

```
#if defined(__cpp_char8_t)
#define U8(x) u8##x##_as_char
#else
#define U8(x) u8##x
#endif
```

Before	After
`constexpr const char &r = u8'x';`	`constexpr const char &r = U8('x');  // C++17 or C++20`
`constexpr const char *p = u8"text";`	`constexpr const char *p = U8("text");  // C++17 or C++20`

`// Standard C++ doesn't permit conversion to arrays of unknown,`
`// bound, but versions of gcc prior to 9.0.1 did.`
`constexpr const char (&r)[] = u8"text";`
	`// If P0388R2 [P0388R2] is adopted, then this will be ok.`
	
	`constexpr const char (&r)[] = U8("text"); // C++17 or C++20 with P038`

An implementation of the above `U8` macro is available in a header only library available at https://github.com/tahonermann/char8_t-remediation.

Substitute class types for C arrays initialized with `u8` string literals

In C++17, arrays of `char` may be initialized with `u8` string literals, but such initialization is ill-formed in C++20. C++17 behavior can be emulated by substituting a class type with appropriate class template argument deduction guides.

```
#include <utility>

template<std::size_t N>
struct char_array {
  template<std::size_t P, std::size_t... I>
  constexpr char_array(
    const char (&r)[P],
    std::index_sequence<I...>)
  :
    data{(I<P?r[I]:'\0')...}
  {}
  template<std::size_t P, typename = std::enable_if_t<(P<=N)>>
  constexpr char_array(const char(&r)[P])
    : char_array(r, std::make_index_sequence<N>())
    {}

#if defined(__cpp_char8_t)
  template<std::size_t P, std::size_t... I>
```

```
  constexpr char_array(
    const char8_t (&r)[P],
    std::index_sequence<I...>)
  :
    data{(I<P?static_cast<char>(r[I]):'\0')...}
  {}
  template<std::size_t P, typename = std::enable_if_t<(P<=N)>>
  constexpr char_array(const char8_t(&r)[P])
    : char_array(r, std::make_index_sequence<N>())
    {}
#endif

  constexpr (&operator const char() const)[N] {
    return data;
  }
  constexpr (&operator char())[N] {
    return data;
  }

  char data[N];
};
template<std::size_t N>
char_array(const char(&)[N]) -> char_array<N>;
#if defined(__cpp_char8_t)
template<std::size_t N>
char_array(const char8_t(&)[N]) -> char_array<N>;
#endif
```

Before	After
`char a[] = u8"text";`	`char_array a = u8"text";  // Ok, initialized with "text\0"`
`constexpr char a[] = u8"text";`	`constexpr char_array a = u8"text";  // Ok, initialized with "text\0"`
`constexpr char a[3] = u8"text";  // ill-formed`	`constexpr char_array<3> a = u8"text"; // ill-formed (too many initializers)`
`constexpr char a[6] = u8"text";`	`constexpr char_array<6> a = u8"text"; // Ok, initialized with "text\0\0"`

An implementation of the above `char_array` class template is available in a header only library available at https://github.com/tahonermann/char8_t-remediation.

Use explicit conversion functions

Explicit conversion functions can be used, in a C++17 compatible manner, to cope with the change of return type to the `std::filesystem::path` member functions when a UTF-8 encoded path is desired in an object of type `std::string`. For example:

```
std::string from_u8string(const std::string &s) {
  return s;
}
std::string from_u8string(std::string &&s) {
  return std::move(s);
}
#if defined(__cpp_lib_char8_t)
std::string from_u8string(const std::u8string &s) {
  return std::string(s.begin(), s.end());
}
#endif

std::filesystem::path p = ...;
std::string s = from_u8string(p.u8string());  // C++17 or C++20
```

This naturally incurs a cost when building with `char8_t` support enabled due to the need to copy the path contents.

Tooling

Tooling could potentially assist programmers in migrating code. Several of the approaches discussed above could be applied mechanically to an existing code base. For example, re-writing existing `u8` literals to ordinary literals with escape sequences, or adding an `_as_char` UDL suffix to existing literals (inserting include directives as needed).

Options considered to reduce backward compatibility impact

The following sections summarize options that have been considered to reduce backward compatibility impact. Most of these options are *not* proposed in this paper because they would actively interfere with goals of the `char8_t` proposal; to enable the type system to protect against inadvertent mixing of UTF-8 data and the execution encoding. However, some of these options may be useful for some code bases and could be provided by implementations as opt-in extensions.

Only two of these options (7 and 8) are proposed for inclusion in the standard. In both of these cases, the concern that is addressed was not specifically intended by the changes adopted in P0482R6. These are effectively bug fixes.

1) Reinstate `u8` literals as type `char` and introduce a new literal prefix for `char8_t`

Not proposed

Many of the backward compatibility concerns could be avoided by reinstating `u8` literals as having type `char` and introducing a new prefix, for example `U8`, to specify UTF-8 literals with type `char8_t`.

The visible difference between `u8` and `U8` is subtle. Some coding compliance standards, such as MISRA, forbid use of identifiers that differ only in case. It has been suggested that C++11's use of `u` and `U` to denote UTF-16 and UTF-32 literals was a mistake because the visual distinction is too subtle. To avoid these subtle visual differences, new literal prefixes such as `utf8`, `utf16`, and `utf32` could be introduced and the old ones deprecated. The downside of these prefixes is, of course, that they are longer.

Implementing this option would continue enabling problems with encoding confusion that we see today. The execution encoding is not UTF-8 on some popular platforms and continuing to use `char` based types for execution encoding and UTF-8 (and other untrusted input or encodings) is a recipe for continued occurrences of mojibake in applications. For platforms that use UTF-8 as the execution encoding, ordinary literals are already UTF-8 encoded. This option would introduce three distinct ways of writing UTF-8 literals on such platforms; having two ways to do (almost) the same thing is usually one too many already.

2) Allow implicit conversions from `char8_t` to `char`

Not proposed

Allowing implicit conversions from `char8_t` to `char` was considered with the original P0482 proposal. The concerns with this approach are the same as in option 1; this enables continued, potentially unintended, mixing of UTF-8 data with non-UTF-8 data resulting in mojibake.

Additionally, allowing implicit conversions would not address all compatibility concerns. For example:

```
template<typename T> void f(T); // #1
void f(char);                   // #2
f(u8'x'); // Calls #2 in C++17, would still call #1 in C++20.
```

However, such implicit conversions could still be useful for some existing code. Implementations could offer extensions to enable such conversions.

3) Allow initializing an array of `char` with a `u8` string literal

Not proposed

This option would allow the following code to remain well-formed in C++20.

```
char a[] = u8"text";
```

Array initialization is the one context in which the previously discussed uses of `reinterpret_cast` or the `_as_char` UDL isn't an option. This option would allow array initializations to remain well-formed and avoid the need for workarounds like the previously discussed `char_array` template. However, this option would continue to promote mixing of UTF-8 data with non-UTF-8 data potentially resulting in mojibake.

Implementations could allow these initializations as a conforming extension.

4) Allow initializing an array with a reference to an array

Not proposed

This option would enable use of the previously discussed `_as_char` UDL to initialize an array without the need for workarounds like the previously discussed `char_array` template. However, this option would continue to promote mixing of UTF-8 data with non-UTF-8 data potentially resulting in mojibake.

```
char a[] = u8"text"_as_char;
```

Implementations could allow these initializations as a conforming extension.

5) Allow `std::string` to be initialized with `char8_t` based types

Not proposed

This option has been suggested as a way to allow some existing uses of `std::string` to hold UTF-8 data to remain valid in C++20. For example:

```
std::string s1 = u8"text";
std::string s2 = s1 + u8"text";
```

This option constitutes a narrow fix for a few specific use cases within a considerably larger problem space. Further, it would require changes to `std::basic_string` specifically for its `char`-based specializations. As with previously discussed options, this would again continue to promote mixing of UTF-8 data with non-UTF-8 data potentially resulting in mojibake.

6) Allow implicit conversions from `std::u8string` to `std::string`

Not proposed

This option has been suggested as a means to address the backward compatibility impact due to the changes to the `std::filesystem::path` `u8string` and `generic_u8string` member functions. It would allow code like the following to continue to work as expected:

```
std::filesystem::path p = ...;
std::string s1 = p.u8string();
```

This option is, again, not proposed because it would allow unintended mixing of UTF-8 encoded data and the execution character encoding.

7) Add deleted ostream inserters for `char8_t`, `char16_t`, and `char32_t`

Proposed

An unintended and silent behavioral change was introduced with the adoption of P0482R6. In C++17, the following code wrote the code units of the literals to stdout. In C++20, this code now writes the character literal as a number, and the address of the string literal, to stdout.

```
std::cout << u8'x';    // In C++20, writes the number 120.
std::cout << u8"text"; // In C++20, writes a memory address.
```

This is a surprising change that provides no benefit to programmers. Adding deleted ostream inserters would avoid this surprising behavioral change while reserving the possibility to specify behavior for these operations in the future (for example, to specify implicit transcoding to the execution encoding).

8) Allow `std::filesystem::u8path` to accept ranges and iterators with `char8_t` value types

Proposed

Another unintended behavioral change introduced with the adoption of P0482R6 is that the following code is now ill-formed because `std::filesystem::u8path` requires a range or pair of iterators specifically with a value type of `char`.

```
std::filesystem::u8path(u8"text");
```

`std::filesystem::u8path` is now deprecated, but since it previously required UTF-8 data, there is no risk of encoding confusion (unlike with many of the other options discussed in this paper). Allowing it to continue to be called with `u8` literals (or other `char8_t` based ranges and iterators) causes no harm other than potentially encouraging continued use of a deprecated interface.

Proposal

This paper proposes implementing only options 7 and 8.

- Add deleted overloads of `basic_ostream<char, ...>::operator<<` and `basic_ostream<wchar_t, ...>::operator<<` for `char8_t` character and string types. This avoids the silent and surprising behavior change introduced by P0482R6 [P0482R6] that resulted in UTF-8 characters being formatted as numeric values and UTF-8 strings being formatted as pointers.
- Add deleted overloads of `basic_ostream<char, ...>::operator<<` for `wchar_t`, `char16_t` and `char32_t` character and string types and deleted overloads of `basic_ostream<wchar_t, ...>::operator<<` for `char16_t` and `char32_t` character and string types. This removes surprising behavior that has been present since C++11; that characters are formatted as numeric values and that strings are formatted as pointers.
- Modify `std::filesystem::u8path` to accept ranges and iterators with `char8_t` value types. This allows existing code that passes UTF-8 string literals to remain well-formed.
  `u8path(u8"filename"); // Ok; ill-formed following P0482R6 [P0482R6].`
- Update the `__cpp_lib_char8_t` feature test macro to reflect proposed changes in library behavior.

Wording

☐ Hide deleted text

These changes are relative to N4820 [N4820]

Library wording

Change in table 36 of 17.3.1 [support.limits.general] paragraph 3:

Table 35 — Standard library feature-test macros

Macro name	Value	Header(s)
[…]	[…]	[…]
__cpp_lib_char8_t	~~201811~~**201907L** ***** placeholder *****	<atomic> <filesystem> <istream> <limits> <locale> <ostream> <string> <string_view>
[…]	[…]	[…]

Drafting note: the final value for the `__cpp_lib_char8_t` feature test macro will be selected by the project editor to reflect the date of approval.

Change in 29.7.5.1 [ostream]:

```
namespace std {

  […]
```

```
  // [ostream.inserters.character], character inserters
  template<class charT, class traits>
    basic_ostream<charT, traits>& operator<<(basic_ostream<charT, traits>&, charT);
  template<class charT, class traits>
    basic_ostream<charT, traits>& operator<<(basic_ostream<charT, traits>&, char);
  template<class traits>
    basic_ostream<char, traits>& operator<<(basic_ostream<char, traits>&, char);

  template<class traits>
    basic_ostream<char, traits>& operator<<(basic_ostream<char, traits>&, signed char);
  template<class traits>
    basic_ostream<char, traits>& operator<<(basic_ostream<char, traits>&, unsigned char);

  // The following deleted overloads prevent formatting character values as numeric values.
  template<class traits>
    basic_ostream<char, traits>& operator<<(basic_ostream<char, traits>&, wchar_t) = delete;
  template<class traits>
    basic_ostream<char, traits>& operator<<(basic_ostream<char, traits>&, char8_t) = delete;
  template<class traits>
    basic_ostream<char, traits>& operator<<(basic_ostream<char, traits>&, char16_t) = delete;
  template<class traits>
    basic_ostream<char, traits>& operator<<(basic_ostream<char, traits>&, char32_t) = delete;
  template<class traits>
    basic_ostream<wchar_t, traits>& operator<<(basic_ostream<wchar_t, traits>&, char8_t) = delete;
  template<class traits>
    basic_ostream<wchar_t, traits>& operator<<(basic_ostream<wchar_t, traits>&, char16_t) = delete;
  template<class traits>
    basic_ostream<wchar_t, traits>& operator<<(basic_ostream<wchar_t, traits>&, char32_t) = delete;

  template<class charT, class traits>
    basic_ostream<charT, traits>& operator<<(basic_ostream<charT, traits>&, const charT*);
  template<class charT, class traits>
    basic_ostream<charT, traits>& operator<<(basic_ostream<charT, traits>&, const char*);
  template<class traits>
    basic_ostream<char, traits>& operator<<(basic_ostream<char, traits>&, const char*);

  template<class traits>
    basic_ostream<char, traits>& operator<<(basic_ostream<char, traits>&, const signed char*);
  template<class traits>
    basic_ostream<char, traits>& operator<<(basic_ostream<char, traits>&, const unsigned char*);

  // The following deleted overloads prevent formatting strings as pointer values.
  template<class traits>
    basic_ostream<char, traits>& operator<<(basic_ostream<char, traits>&, const wchar_t*) = delete;
  template<class traits>
    basic_ostream<char, traits>& operator<<(basic_ostream<char, traits>&, const char8_t*) = delete;
  template<class traits>
    basic_ostream<char, traits>& operator<<(basic_ostream<char, traits>&, const char16_t*) = delete;
  template<class traits>
    basic_ostream<char, traits>& operator<<(basic_ostream<char, traits>&, const char32_t*) = delete;
  template<class traits>
    basic_ostream<wchar_t, traits>& operator<<(basic_ostream<wchar_t, traits>&, const char8_t*) = delete;
  template<class traits>
    basic_ostream<wchar_t, traits>& operator<<(basic_ostream<wchar_t, traits>&, const char16_t*) = delete;
  template<class traits>
    basic_ostream<wchar_t, traits>& operator<<(basic_ostream<wchar_t, traits>&, const char32_t*) = delete;
}
```

Annex C Compatibility wording

Change in C.5.11 [diff.cpp17.input.output] paragraph 2:

Affected subclause: 29.7.5.2.4
Change: Overload resolution for ostream inserters used with UTF-8 literals.
Rationale: Required for new features.
Effect on original feature: Valid ISO C++ 2017 code that passes UTF-8 literals to `basic_ostream<char, ...>::operator<<` **or** `basic_ostream<wchar_t, ...>::operator<<` ~~no longer calls character related overloads~~**is now ill-formed**.

```
  std::cout << u8"text";       // Previously called operator<<(const char*) and printed a string.
                               // Now ~~calls operator<<(const void*) and prints a pointer value~~ill-formed.
  std::cout << u8'X';          // Previously called operator<<(char) and printed a character.
                               // Now ~~calls operator<<(int) and prints an integer value~~ill-formed.
```

Add a new paragraph after C.5.11 [diff.cpp17.input.output] paragraph 2:

Affected subclause: 29.7.5.2.4
Change: Overload resolution for ostream inserters used with `wchar_t`, `char16_t`, or `char32_t` types.
Rationale: Removal of surprising behavior.
Effect on original feature: Valid ISO C++ 2017 code that passes `wchar_t`, `char16_t`, or `char32_t` characters or strings to `basic_ostream<char, ...>::operator<<`, or that passes `char16_t`, or `char32_t` characters or strings to `basic_ostream<wchar_t, ...>::operator<<` is now ill-formed.

```
std::cout << u"text";         // Previously formatted the string as a pointer value.
                              // Now ill-formed.
std::cout << u'X';            // Previously formatted the character as an integer value.
                              // Now ill-formed.
```

Annex D Compatibility features wording

Change in D.17 [depr.fs.path.factory] paragraph 1:

```
template<class Source>
  path u8path(const Source& source);
template<class InputIterator>
  path u8path(InputIterator first, InputIterator last);
```

Requires: The `source` and `[first, last)` sequences are UTF-8 encoded. The value type of `Source` and `InputIterator` is `char` **or `char8_t`**. `Source` meets the requirements specified in 29.11.7.3.

References

[N4820] "Working Draft, Standard for Programming Language C++", N4820, 2019.
http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/n4820.pdf

[P0388R2] Robert Haberlach, "Permit conversions to arrays of unknown bound", P0388R2, 2018.
http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0388r2.html

[P0482R6] Tom Honermann, "char8_t: A type for UTF-8 characters and strings (Revision 6)", P0482R6, 2018.
http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0482r6.html

[P0732R2] Jeff Snyder and Louis Dionne, "Class Types in Non-Type Template Parameters", P0732R2, 2018.
http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0732r2.pdf

[SD-8] Titus Winters, "SD-8: Standard Library Compatibility", SD-8, 2018.
https://isocpp.org/std/standing-documents/sd-8-standard-library-compatibility